实验一

进程与线程——Linux 进程与线程通信

1.1 实 验 目 的

深刻理解线程和进程的概念，掌握线程与进程在组成成分上的差别，以及与其相适应的通信方式和应用目标。

Linux 系统的 fork( )保持了 UNIX 的经典语义，被创建进程具有独立于父进程的地址空间，二者之间的通信通常可以采用 pipe 机制。clone( )是 Linux 系统特有的系统调用，可以通过参数确定父子之间是否共享存储空间等资源，在地址空间等资源共享的情况下 clone( )实质上相当于创建了一个轻进程或线程，这是 clone( )的通常用法。实际在 Linux 系统中，fork( )以及用户级线程 pthread 都是基于 clone( )实现的。

1.2 实 验 内 容

以 Linux 系统进程和线程机制为背景，掌握 fork( )和 clone( )系统调用的形式和功能，以及与其相适应的高级通信方式。由 fork( )派生的子进程之间通过 pipe 通信，由 clone( )创建的线程之间通过共享内存通信，对于后者需要考虑互斥问题。

以生产者 - 消费者问题为例，通过实验理解 fork( )和 clone( )两个系统调用的区别。程序要求能够创建 4 个进程或线程，其中包括两个生产者和两个消费者，生产者和消费者之间能够传递数据。

1.3 实 验 准 备

1. fork( )系统调用

```
pid = fork( );
```

创建一个子进程，子进程是父进程的完整复制，正常返回值为非负整数，对于父进程来说该数大于 0，是子进程的编号(pid)；对于子进程来说该数为 0。正是利用返回值的差别来决定二者不同的后继动作。

2. clone( )系统调用

```
int clone(int( * fn)(void * arg),void * stack,int flags,void * arg);
```

其中 fn 是轻进程所执行的函数，stack 是轻进程所使用的栈，flag 是 CLONE_VM，CLONE_FS，

CLONE_FILES,CLONE_SIGHAND,CLONE_PID 的组合,arg 是调用过程的对应参数。clone()的关键是 flag 的设定,CLONE_VM 表示子进程共享父进程内存,CLONE_FS 表示子进程共享父进程的文件系统,CLONE_SIGHAND 表示子进程共享父进程的消息处理机制,CLONE_PID 是指子进程继承父进程的 id 号。

3. pipe()系统调用

ret_val = pipe(fd);

参数定义为 int fd[2]。创建一个管道文件,返回两个文件描述符 fd[0]和 fd[1],分别用于管道文件的读和写操作。管道文件创建后,可以被 fork()创建的子进程共享。

4. sem_wait(&s)和 sem_post(&s)分别相当于信号量的 P 操作和 V 操作。其中 s 是说明为 sem_t 类型的信号量。初始化函数为 sem_init(s,0,8)。

5. pthread_mutex_lock(&mutex)和 pthread_mutex_unlock(&mutex)分别用于加锁和解锁。参数为 pthread_mutex_t mutex 定义的互斥锁。初始化函数为 tthread_mutex_init(&mutex,NULL)。

1.4 实验设计

1. 用 pipe()创建一个管道文件,然后用 fork()创建 2 个生产进程和 2 个消费进程,它们之间通过 pipe()传递信息。

2. 用 clone()创建 4 个轻进程(线程),用参数指明共享内存等资源,通过共享内存模拟生产者-消费者问题,利用 pthread_mutex_lock(),pthread_mutex_unlock()等函数实现对共享存储区访问的互斥。

1.5 参考代码

1. 基于 fork()系统调用

```
#include "sys/types.h"
#include "sys/file.h"
#include "unistd.h"

char r_buf[4];   /*读缓冲*/
char w_buf[4];   /*写缓冲*/
int pipe_fd[2];
pid_t pid1,pid2,pid3,pid4;

int producer(int id);
```

```
int consumer(int id);

int main(int argc,char ** argv)
{
    if(pipe(pipe_fd) <0)
    {
        printf("pipe create error \n");
        exit(-1);
    }
    else
    {
        printf("pipe is created successfully! \n");
        if((pid1 =fork()) ==0)
          producer(1);
        if((pid2 =fork()) ==0)
          producer(2);
        if((pid3 =fork()) ==0)
          consumer(1);
        if((pid4 =fork()) ==0)
          consumer(2);
    }
    close(pipe_fd[0]);  /*需要加上这两句*/
    close(pipe_fd[1]);  /*否则会有生产者或者消费者永远等待*/

    int i,pid,status;
    for(i =0;i <4;i ++)
       pid =wait(&status);
    exit(0);
}

int producer(int id)
{
    printf("producer %d is running! \n",id);
    close(pipe_fd[0]);
    int i =0;
```

```
    for(i=1;i<10;i++)
    {
        sleep(3);
        if(id==1) /*生产者1*/
            strcpy(w_buf,"aaa\0");
        else  /*生产者2*/
            strcpy(w_buf,"bbb\0");
        if(write(pipe_fd[1],w_buf,4)==-1)
            printf("write to pipe error\n");
    }
    close(pipe_fd[1]);
    printf("producer %d is over! \n",id);
    exit(id);
}

int consumer(int id)
{
    close(pipe_fd[1]);
    printf("producer %d is running! \n",id);
    if (id==1)  /*消费者1*/
        strcpy(w_buf,"ccc\0");
    else  /*消费者2*/
        strcpy(w_buf,"ddd\0");
    while(1)
    {
        sleep(1);
        strcpy(r_buf,"eee\0");
        if(read(pipe_fd[0],r_buf,4)==0)
            break;
        printf("consumer %d get %s,while the w_buf is %s\n",id,r_buf,w_buf);
    }
    close(pipe_fd[0]);
    printf("consumer %d is over! \n",id);
    exit(id);
}
```

2. 基于 clone( )系统调用

```
#include "sched.h"
#include "pthread.h"
#include "stdio.h"
#include "stdlib.h"
#include "semaphore.h"

int producer(void * args);
int consumer(void *args);
pthread_mutex_t mutex;
sem_t product;
sem_t warehouse;

char buffer[8][4];
int bp=0;

main(int argc,char ** argv)
{
    pthread_mutex_init(&mutex,NULL);
    sem_init(&product,0,0);
    sem_init(&warehouse,0,8);
    int clone_flag,arg,retval;
    char *stack;
    clone_flag=CLONE_VM|CLONE_SIGHAND|CLONE_FS|CLONE_FILES;
    int i;
    for(i=0;i<2;i++)
    {  /*创建4个线程*/
        arg=i;
        stack=(char*)malloc(4096);
        retval=clone((void*)producer,&(stack[4095]),clone_flag,(void*)&arg);
        stack=(char*)malloc(4096);
        retval=clone((void*)consumer,&(stack[4095]),clone_flag,(void*)&arg);
        sleep(1);
    }
    exit(1);
```

```
}

int producer(void * args)
{
    int id = *((int *)args);
    int i;
    for(i=0;i<10;i++)
    {
        sleep(i+1);  /*表现线程速度差别*/
        sem_wait(&warehouse);
        pthread_mutex_lock(&mutex);
        if(id==0)
            strcpy(buffer[bp],"aaa\0");
        else
            strcpy(buffer[bp],"bbb\0");
        bp++;
        printf("producer%d produce %s in %d\n",id,buffer[bp-1],bp-1);
        pthread_mutex_unlock(&mutex);
        sem_post(&product);
    }
    printf("producer%d is over! \n",id);
}

int consumer(void *args)
{

    int id = *((int *)args);
    int i;
    for(i=0;i<10;i++)
    {
        sleep(10-i);  /*表现线程速度差别*/
        sem_wait(&product);
        pthread_mutex_lock(&mutex);
        bp--;
        printf("consumer%d get %s in %d\n",id,buffer[bp],bp);
```

```
            strcpy(buffer[bp],"zzz\0");
            pthread_mutex_unlock(&mutex);
            sem_post(&warehouse);
        }
        printf("consumer%d is over! \n",id);
}
```

1.6 实验结果

程序1的输出如下:

```
pipe is created successfully!
producer 1 is running!
producer 2 is running!
producer 1 is running!
producer 2 is running!
consumer 2 get aaa,while the w_buf is ddd
consumer 1 get bbb,while the w_buf is ccc
consumer 1 get aaa,while the w_buf is ccc
consumer 2 get bbb,while the w_buf is ddd
consumer 2 get aaa,while the w_buf is ddd
consumer 1 get bbb,while the w_buf is ccc
consumer 1 get aaa,while the w_buf is ccc
consumer 2 get bbb,while the w_buf is ddd
consumer 2 get aaa,while the w_buf is ddd
consumer 1 get bbb,while the w_buf is ccc
consumer 1 get aaa,while the w_buf is ccc
consumer 2 get bbb,while the w_buf is ddd
consumer 2 get aaa,while the w_buf is ddd
consumer 1 get bbb,while the w_buf is ccc
consumer 1 get aaa,while the w_buf is ccc
consumer 2 get bbb,while the w_buf is ddd
producer 1 is over!
producer 2 is over!
consumer 2 get aaa,while the w_buf is ddd
consumer 1 get bbb,while the w_buf is ccc
```

```
consumer 2 is over!
consumer 1 is over!
```

程序2的输出如下

```
producer0 produce aaa in 0
producer1 produce bbb in 1
producer0 produce aaa in 2
producer1 produce bbb in 3
producer0 produce aaa in 4
producer1 produce bbb in 5
consumer0 get bbb in 5
consumer1 get aaa in 4
producer0 produce aaa in 4
producer1 produce bbb in 5
producer0 produce aaa in 6
producer1 produce bbb in 7
consumer0 get bbb in 7
consumer1 get aaa in 6
producer0 produce aaa in 6
producer1 produce bbb in 7
consumer0 get bbb in 7
consumer1 get aaa in 6
producer0 produce aaa in 6
producer1 produce bbb in 7
consumer0 get bbb in 7
consumer1 get aaa in 6
producer0 produce aaa in 6
producer1 produce bbb in 7
consumer0 get bbb in 7
consumer1 get aaa in 6
consumer0 get bbb in 5
consumer1 get aaa in 4
producer0 produce aaa in 4
producer1 produce bbb in 5
consumer0 get bbb in 5
```

```
consumer1 get aaa in 4
consumer0 get bbb in 3
consumer1 get aaa in 2
consumer0 get bbb in 1
consumer1 get aaa in 0
producer0 produce aaa in 0
producer0 is over!
producer1 produce bbb in 1
producer1 is over!
consumer0 get bbb in 1
consumer0 is over!
consumer1 get aaa in 0
consumer1 is over!
```

分析程序1结果可见,当一个进程改变其空间数据时,其他进程空间对应的数据内容并未改变,说明使用fork()语句创建的子进程与其父进程具有相对独立的地址空间,在解决生产消费的问题时,可以采用pipe()进行通信。因为子进程复制了父进程的打开文件表,所以pipe()所建立的通信管道可被子进程继承,生产和消费进程可以通过对同一管道文件的读写进行通信。

程序1中,消费者从管道中接收生产者发送的数据,并且和自己存储区中的数据进行比较,两者的数据是不同的,说明两个进程拥有不同的存储空间。

分析程序2结果可见,clone()语句在创建进程时,可通过参数设定子进程与父进程是否共享存储空间,从而可以创建真正意义上的线程。生产者和消费者进程共享内存,从而可以通过共享内存直接交换数据。但是多个进程共享内存需要互斥机制,程序中定义了临界区变量mutex和两个信号量product,warehouse,临界区变量用于共享内存操作的互斥,信号量分别实现了生产者和消费者的等待。

程序2中,消费者输出存储区中的数据,并且存储区中的数据根据生产者存入的数据而发生变化,说明clone()语句通过flags的设定实现了共享内存。若在实验中除去CLONE_VM选项,将出现非预期的结果。

1.7 思考问题

1. 在Linux环境中,除pipe和clone公共内存通信外,还可以采用shm和msg通信方式,试查阅相关资料了解这些通信方式,并使用上述任意一种方式模拟实现生产者-消费者问题。

2. 比较Linux系统中pipe、clone、shm和msg四种高级通信方法的优缺点以及各自适应的环境。